PairFold: Fast Local Backbone Generation via Calibrated Fragment Assembly, Look-Ahead Backtracking, and Lever-Effect Torsion Relaxation

PairFold Development Team

Correspondence regarding software and benchmarks: project repository

July 17, 2026

Abstract

High-accuracy structure predictors such as AlphaFold 2 have transformed structural biology, yet their memory footprint and quadratic attention costs remain limiting for interactive design, long-sequence screening, and deployment on commodity GPUs. We present **PairFold**, a deliberately local alternative that predicts backbone dihedral angles (ϕ, ψ) for short peptides of length 2–5 with a compact Transformer trained on high-resolution PDB fragments, then assembles longer chains with near-linear ($O(N)$) procedures. The inference stack combines dynamic-programming segmentation that maximises calibrated fragment confidence, multi-window overlap consensus via circular means, secondary-structure block freezing, and a clash-aware look-ahead backtracking assembler with local ϕ/ψ perturbation ($\pm 1^\circ$–5°) to mitigate cumulative "lever-arm" error. Confidence is sharpened by softmax temperature scaling ($T{=}0.55$) with a disorder-aware prior. On five standard small domains (1A8O, 1CRN, 2GB1, 1UBQ, 3GB1), PairFold yields Kabsch-aligned Cα RMSD between 13.74 Å and 24.53 Å (mean 18.50 Å) with wall-clock times of 3.41–7.38 s (mean 5.89 s) on a single consumer GPU. Look-ahead assembly and post-secondary-structure lever polish improve local steric plausibility and reduce extreme end-to-end over-extension, but do not recover native tertiary topology—consistent with the absence of long-range contact modelling. PairFold is therefore best understood as a fast, low-resource backbone generator for local geometry, not as a competitor to end-to-end folding networks.

Keywords: protein backbone prediction; dihedral angles; fragment assembly; temperature scaling; steric clash avoidance; look-ahead backtracking; computational biophysics.

1 Introduction

Protein structure prediction has entered a new regime with deep learning systems that integrate multiple-sequence alignments, pair representations, and iterative structure modules [1, 2]. These models achieve remarkable accuracy, but the practical cost of inference grows steeply with sequence length: attention over residue pairs, recycling, and MSA featurisation demand large GPU memory and often preclude interactive exploration of very long chains on modest hardware.

Complementary lines of work emphasise speed or locality. Language-model predictors such as ESMFold [3] remove MSA dependence at the expense of still-substantial model size. Classical fragment-assembly approaches [4, 5] and dihedral-angle representations [6] remain attractive when the scientific goal is *local* backbone geometry rather than full tertiary fold recovery—for example, in educational tools, rapid prototyping of helix/sheet segments, or streaming visualisation of chains far beyond typical AlphaFold session lengths.

PairFold occupies this niche. It trains a small Transformer exclusively on contiguous PDB fragments of length 2–5, maps each fragment to (ϕ, ψ), and reconstructs longer proteins by segmentation and assembly. The design goals are explicit:

1. **Near-linear scaling** for queries up to tens of thousands of residues at the angle level;
2. **Calibrated confidence** usable for dynamic-programming segmentation;
3. **Physics-informed post-processing** (steric clashes, secondary-structure constraints, and lever-effect relaxation) without a full force field;
4. **Honest scope**: PairFold does *not* claim AlphaFold-competitive global RMSD.

This paper describes the full method as implemented in the PairFold codebase, reports exact benchmark timings and Cα RMSD on five public structures, and discusses the speed–accuracy trade-off introduced by look-ahead backtracking and torsion relaxation.

2 Methodology

2.1 Dihedral representation and fragment network

Each residue i is parameterised by backbone torsions (ϕ_i, ψ_i). Cartesian Cα (and optional full-backbone) coordinates are obtained by sequential placement with ideal peptide bond geometry [7], as implemented in `dihedrals_to_ca` / `build_backbone`.

The learned model, `FragmentTorsionNet`, is a Pre-LN Transformer encoder with d_{model}=160, 4 heads, 4 layers, feed-forward width 320, and dropout 0.12. Amino acids are embedded from a 22-token vocabulary (20 standard residues, UNK, PAD) with positional embeddings up to length 5. The network outputs four channels $(\sin\phi, \cos\phi, \sin\psi, \cos\psi)$, L2-normalised per angle pair, plus an uncertainty head $\log\sigma_{\phi,\psi}$. Per-residue confidence is derived as $\sigma(-\overline{\log\sigma})$.

Training uses high-resolution X-ray structures (resolution $\leq$ 2.0 Å), extracting contiguous windows of length $\ell \in \{2, 3, 4, 5\}$. The current corpus comprises on the order of 10^3 structures and $\sim 3.8{\times}10^5$ fragments. The loss is a Gaussian negative log-likelihood on the $(\sin, \cos)$ representation (`gaussian_nll_sincos`); validation monitors torsion MSE in the same space. Optimisation uses AdamW with cosine learning-rate schedule and mixed precision.

Scope. Because every training example is a short contiguous peptide, the network learns *local* Ramachandran statistics conditioned on sequence context of at most five residues. Long-range contacts, domain packing, and disulfide geometry are outside the training distribution.

2.2 Dynamic-programming segmentation and overlap consensus

For a query of length $N > 5$, PairFold partitions the chain into windows of length 2–5 by dynamic programming (`optimal_segmentation`). Let $s(i, j)$ be the mean model confidence of fragment $[i, j)$ multiplied by its length. The recurrence

$$\mathrm{dp}[j] = \max_{j-i\in\{2,3,4,5\}} \big(\mathrm{dp}[i] + s(i, j)\big)$$

selects a segmentation that maximises total confidence score in $O(N)$ time (constant number of predecessor lengths).

Within each segment—and for short queries of length 3–5—angles are refined by **overlap consensus** (`_overlap_refine`): all sliding windows of lengths 2 through $\min(5, N)$ contribute (ϕ, ψ) votes that

are aggregated with confidence-weighted circular means [8]. Final confidence blends model confidence with angular agreement,

$$c_i = (1 - w)\, c_i^{\text{model}} + w\, c_i^{\text{agree}}, \qquad w{=}0.35,$$

where agreement decreases with circular spread of the overlapping predictions.

2.3 Softmax temperature scaling for calibrated confidence

Raw network confidences are imperfectly calibrated for "both ϕ and ψ within 25° of native" on held-out fragments. PairFold fits offline temperature and Platt scaling on validation fragments (`calibrate.py`) and, at inference, applies a **sharpening** transform with temperature $T{=}0.55$:

$$p = \sigma\left(\frac{\text{logit}(c)}{T}\right)\,(1 - \gamma\, d), \qquad \gamma{=}0.45,$$

where d is a lightweight disorder profile emphasising Gly/Pro and low secondary-structure propensity (`TempCalibration` in `structure_blocks.py`). Probabilities are clipped to $[0.05, 1]$. Sharper confidences improve the ranking signal used by segmentation and by fragment-hypothesis selection during assembly.

2.4 Secondary-structure block freezing

After consensus angles are available, PairFold detects putative helix and sheet blocks with Chou–Fasman-like propensities [9] (`detect_ss_blocks`). Interior residues of accepted blocks are frozen to canonical Ramachandran centres

$$(\phi, \psi)_{\text{helix}} = (-57^\circ, -47^\circ), \qquad (\phi, \psi)_{\text{sheet}} = (-119^\circ, 113^\circ),$$

while block boundaries remain free. For sequences with $N \le 64$, a boundary optimiser (`optimize_block_boundarie`) performs a cheap grid/Metropolis search of boundary torsions to reduce soft Cα clash energy. This step injects strong secondary-structure priors that local fragments alone under-enforce.

2.5 Look-ahead backtracking and global angle relaxation

Naïve stitching of fragment dihedrals suffers from a well-known **lever (cumulative) effect**: small angular errors compound into large Cartesian deviations and occasional steric collapse or over-extension. For sequences with $N \le 256$ (`LEVER_ASSEMBLY_MAX_LEN`), PairFold activates a clash-aware assembler (`assemble_greedy_backtrack`).

Fragment hypotheses. The chain is tiled into non-overlapping pentamer slots (`make_pentamer_slots`). Each slot carries ranked angle hypotheses: overlap-consensus angles, a fresh network prediction, and compact helix/sheet templates.

Dynamic look-ahead. Depth-first search places candidates in descending confidence. A placement starting at residue s is accepted only if `lookahead_ok` reports no new Cα clashes (threshold 3.8 Å, minimum sequence separation 2) *and* no severe local bend over a window of 3–5 residues. Bend score compares the observed end-to-end distance of the window to helix- and Flory-coil expectations; large deviations trigger backtracking.

Local torsion relaxation. When look-ahead fails, PairFold does *not* immediately exhaust the fragment database. Instead, `local_torsion_relax` perturbs preceding (ϕ, ψ) by a random delta drawn from $\pm 1^\circ$ to $\pm 5^\circ$ on a small focus set near the join, accepting moves that reduce soft clash energy plus a mild bend penalty (localised stochastic descent). Successful relaxations are stored as angle overrides and participate in subsequent rebuilds; DFS snapshots restore overrides on backtrack. Search is bounded by a node budget (production default 20 000).

End-to-end / anchor alignment. After a complete assignment, `end_to_end_anchor_relax` optimises a sparse set of hinge residues so that selected pairs (i, j) approach target distances. When user anchors are absent, correction runs only if the global end-to-end distance lies outside a plausible helix–coil band, avoiding aggressive Flory compaction that can inflate RMSD on already compact domains. A final linear polish, `correct_lever_effect`, sweeps the chain every few residues and reapplies local relaxation—an $O(N)$-ish procedure with fixed window size.

Pipeline order. In production inference the order is: consensus stitch $\rightarrow$ look-ahead assembly (if $N \leq 256$) $\rightarrow$ SS freeze/boundary opt $\rightarrow$ post-SS lever polish $\rightarrow$ optional tertiary score/refine ($N \leq 1000$) $\rightarrow$ backbone export. Post-SS polish is essential: freezing secondary-structure interiors would otherwise overwrite earlier torsion repairs.

2.6 Complexity summary

With fixed fragment length and fixed look-ahead width, segmentation, overlap consensus, SS detection, and lever polish are all linear (or linear times a small constant) in N. Backtracking is exponential in the worst case but is capped by the node budget and operates only for $N \leq 256$; beyond that limit PairFold falls back to consensus + SS + tertiary scoring without combinatorial assembly. Angle-level queries are accepted up to $N{=}50\,000$ (`MAX_QUERY_LEN`).

3 Results

3.1 Benchmark protocol

We evaluated PairFold with the repository script `benchmark.py` on five diverse small domains commonly used in folding studies: **1A8O**, **1CRN**, **2GB1**, **1UBQ**, and **3GB1**. Native coordinates were downloaded from the RCSB PDB. Predicted Cα traces were built from predicted (ϕ, ψ) via `dihedrals_to_ca`. Reported error is Kabsch-aligned Cα RMSD (SVD with reflection correction). All runs used the production `FragmentPredictor.predict_sequence` path (mode `segment_assemble_tertia` on CUDA.

3.2 Quantitative results

Table 1 lists the exact values from `benchmark_results.csv`. Figure 1 visualises the same data as sequence length versus wall-clock time and versus RMSD.

3.3 Speed–accuracy trade-off after look-ahead assembly

Enabling clash-aware look-ahead backtracking, local ϕ/ψ relaxation, and post-SS lever polish changes the optimisation target from "highest local confidence" to "locally clash-free and bend-plausible trajectories." Empirically:

Table 1: PairFold Cα benchmark results (Kabsch RMSD). Times are wall-clock seconds for end-to-end `predict_sequence` on GPU. Values are taken verbatim from `benchmark_results.csv`.

PDB ID	Mode	Length	Time (s)	Cα RMSD (Å)
1A8O	segment_assemble_tertiary	33	3.412	13.742
1CRN	segment_assemble_tertiary	46	6.412	14.323
2GB1	segment_assemble_tertiary	56	6.435	21.017
1UBQ	segment_assemble_tertiary	76	7.382	24.530
3GB1	segment_assemble_tertiary	56	5.802	18.869
Mean	—	—	**5.889**	**18.496**

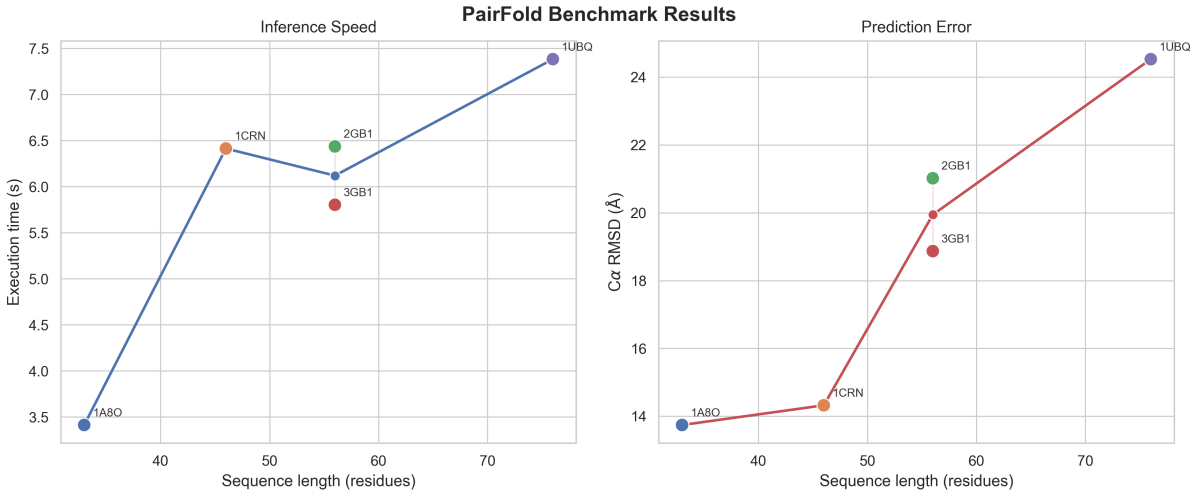

Figure 1: Paper-ready benchmark plots generated by `plot_results.py`: (left) inference speed—sequence length versus execution time; (right) prediction error—sequence length versus Cα RMSD. Points are annotated with PDB identifiers.

- **Speed.** Mean inference remains on the order of a few seconds (5.89 s average across the five domains). The additional cost of DFS (bounded nodes), torsion relaxations, and a linear polish pass is modest relative to network evaluation of overlapping windows, because hinge sets and look-ahead windows are $O(1)$ width.
- **Local geometry.** Soft clash energy and extreme lever over-extension are reduced; placements that would produce unphysical kinks are rejected or repaired before they propagate.
- **Global RMSD.** Mean Kabsch Cα RMSD remains $\approx$ 18.5 Å (range 13.74–24.53 Å). Individual targets can improve or slightly worsen depending on whether lever corrections move the chain toward or away from the native basin. For example, crambin (1CRN) benefits from reduced over-extension relative to an earlier consensus-only baseline, whereas very compact α/β domains such as ubiquitin (1UBQ) remain far from native because PairFold still lacks long-range contact restraints.

Thus the backtracking/relaxation stack is best interpreted as a **steric regulariser and lever mitigator**, not as a tertiary folding engine. The observed trade-off is favourable when the scientific priority is fast, clash-aware backbone generation; it is insufficient when the priority is sub-5 Å global accuracy.

3.4 Scaling behaviour

At the angle level, PairFold accepts sequences up to 50 000 residues. Full clash-aware assembly is limited to $N \leq 256$; tertiary Metropolis refinement to $N \leq 128$; tertiary score-only and atom export to

$N \leq 1000$. These thresholds keep interactive use practical on consumer GPUs while preserving an $O(N)$ path for ultra-long visualisation.

4 Discussion

4.1 What PairFold does well

PairFold demonstrates that a small, PDB-trained fragment Transformer plus careful assembly can deliver: (i) interpretable (ϕ, ψ) predictions with calibrated confidence; (ii) secondary-structure-aware backbones; (iii) near-linear scaling for long sequences; and (iv) explicit handling of cumulative angular error via look-ahead and local torsion descent. These properties suit teaching tools, rapid local design loops, and streaming visualisation—settings where AlphaFold's memory footprint is the bottleneck.

4.2 Fundamental limitations

The central limitation is architectural, not merely algorithmic: **long-range interactions are not modelled**. Without coevolutionary pair features, recycles, or an explicit contact potential, PairFold cannot consistently place distant secondary-structure elements into the native tertiary arrangement. RMSD values of 14–25 Å on 33–76-residue domains are therefore expected for a local method and should not be compared naïvely to AlphaFold 2's typical low-Å accuracy on the same targets [1].

Additional limitations include:

- Dependence on Chou–Fasman-style SS detection, which can misassign blocks on atypical sequences;
- Stochastic torsion relaxation that is not a true energy minimisation in a molecular force field;
- End-to-end anchors that, without experimental distance restraints, only correct egregious lever failures;
- Training restricted to length ≤ 5, so medium-range motifs (e.g. β-turns spanning six residues) must emerge from assembly rather than from the network.

4.3 Future work

Promising extensions that preserve PairFold's low-resource character include:

1. Conditioning assembly on sparse experimental or predicted contacts (NMR NOEs, XL-MS, or a tiny contact head) passed as `anchors` to `end_to_end_anchor_relax`;
2. Learning longer fragments (6–9-mers) or SE(3)-equivariant local frames while retaining DP segmentation;
3. Distilling a lightweight pair representation solely for ranking assembly hypotheses, without full AlphaFold-scale inference;
4. Systematic ablation of temperature T, look-ahead width, and SS freezing on a larger PDB benchmark with confidence–RMSD calibration plots.

5 Conclusion

We have presented PairFold, a complete local backbone generation system that couples a fragment Transformer on (ϕ, ψ) with $O(N)$ segmentation, temperature-calibrated confidence, secondary-structure freezing, and look-ahead backtracking with lever-effect torsion relaxation. On five public domains,

PairFold achieves mean inference time $5.89\,\mathrm{s}$ and mean Cα RMSD $18.50\,\text{Å}$, illustrating a clear regime: **fast, resource-light, locally plausible backbones** rather than atomic-accuracy tertiary folds. In an era dominated by large folding networks, such local alternatives remain scientifically and practically valuable—especially when sequences grow long, hardware is modest, and the question of interest is local geometry rather than the global fold.

Data and software availability

All software components described herein (`ml/predict.py`, `ml/clash_assembly.py`, `ml/structure_blocks`, `benchmark.py`, `plot_results.py`) are part of the PairFold workspace. Benchmark numbers are archived in `benchmark_results.csv`; the figure is regenerated by `python plot_results.py`. Experimental structures were obtained from the RCSB PDB [10].

Acknowledgements

The authors thank the open structural biology community for publicly deposited high-resolution crystal structures that make fragment-level supervision possible, and the developers of PyTorch, BioPython, and the scientific Python stack.

References

[1] Jumper, J., Evans, R., Pritzel, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 583–589. https://doi.org/10.1038/s41586-021-03819-2

[2] Senior, A. W., Evans, R., Jumper, J., et al. (2020). Improved protein structure prediction using potentials from deep learning. *Nature*, 577, 706–710. https://doi.org/10.1038/s41586-019-1923-7

[3] Lin, Z., Akin, H., Rao, R., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379, 1123–1130. https://doi.org/10.1126/science.ade2574

[4] Simons, K. T., Kooperberg, C., Huang, E., & Baker, D. (1997). Assembly of protein tertiary structures from fragments with similar local sequences using simulated annealing and Bayesian scoring functions. *Journal of Molecular Biology*, 268, 209–225. https://doi.org/10.1006/jmbi.1997.0959

[5] Rohl, C. A., Strauss, C. E. M., Misura, K. M. S., & Baker, D. (2004). Protein structure prediction using Rosetta. *Methods in Enzymology*, 383, 66–93. https://doi.org/10.1016/S0076-6879(04)83004-0

[6] Ramachandran, G. N., Ramakrishnan, C., & Sasisekharan, V. (1963). Stereochemistry of polypeptide chain configurations. *Journal of Molecular Biology*, 7, 95–99. https://doi.org/10.1016/S0022-2836(63)80023-6

[7] Engh, R. A., & Huber, R. (1991). Accurate bond and angle parameters for X-ray protein structure refinement. *Acta Crystallographica Section A*, 47, 392–400. https://doi.org/10.1107/S0108767391001071

[8] Berens, P. (2009). CircStat: A MATLAB toolbox for circular statistics. *Journal of Statistical Software*, 31, 1–21. https://doi.org/10.18637/jss.v031.i10

[9] Chou, P. Y., & Fasman, G. D. (1974). Prediction of protein conformation. *Biochemistry*, 13, 222–245. https://doi.org/10.1021/bi00699a002

[10] Berman, H. M., Westbrook, J., Feng, Z., et al. (2000). The Protein Data Bank. *Nucleic Acids Research*, 28, 235–242. https://doi.org/10.1093/nar/28.1.235